



Projektowanie i analiza algorytmów

Porównanie efektywności algorytmów sortowania przez scalanie, quicksort i kubełkowego na rzeczywistym zbiorze danych

Piotr Frąc

Kwiecień 2026

1 Wprowadzenie

Niniejszy projekt dotyczy analizy efektywności trzech algorytmów sortowania: sortowania przez scalanie (merge sort), sortowania szybkiego (quicksort) oraz sortowania kubełkowego (bucket sort).

Sortowanie przez scalanie (Merge Sort)

Merge sort jest algorytmem opartym na paradygmacie „dziel i zwyciężaj”. Tablica wejściowa jest rekurencyjnie dzielona na dwie równe połowy aż do uzyskania tablic jednoelementowych, które są z definicji posortowane. Następnie podtablice są scalane parami w sposób zachowujący porządek elementów.

Przypadek	Złożoność
Pesymistyczny	$O(n \log n)$
Średni	$O(n \log n)$
Optymistyczny	$O(n \log n)$

Sortowanie szybkie (Quicksort)

Quicksort również wykorzystuje paradygmat „dziel i zwyciężaj”. Wybierany jest element osiowy (pivot), względem którego tablica jest dzielona na dwie części: elementy mniejsze od pivota i elementy większe lub równe. Proces jest następnie rekurencyjnie powtarzany dla obu podtablic. W implementacji użyto pivota jako elementu środkowego tablicy, co zmniejsza ryzyko wystąpienia przypadku pesymistycznego.

Przypadek	Złożoność
Pesymistyczny	$O(n^2)$
Średni	$O(n \log n)$
Optymistyczny	$O(n \log n)$

Sortowanie kubełkowe (Bucket Sort)

Bucket sort jest algorytmem nieporównawczym, co oznacza że nie opiera się na wzajemnym porównywaniu elementów. Zamiast tego wykorzystuje znajomość zakresu war-

tości danych. Elementy są rozdzielane do kubeków odpowiadających poszczególnym wartościom, a następnie kubki są składane w posortowaną sekwencję.

Przypadek	Złożoność
Pesymistyczny	$O(n + k)$
Średni	$O(n + k)$
Optymistyczny	$O(n + k)$

Zestawy danych

Testy przeprowadzono dla czterech rozmiarów danych:

Rozmiar	Opis
10 000	Mały zestaw
100 000	Średni zestaw
500 000	Duży zestaw
962 903	Maksymalny rozmiar

2 Metodyka badań

Eksperymenty przeprowadzono na zbiorze danych IMDb zawierającym oceny filmów w postaci liczb całkowitych z zakresu 0–10. Dane wczytano z pliku CSV, pomijając wpisy z pustym polem oceny. Łącznie załadowano 962 903 rekordów.

Struktury danych

Każdy rekord reprezentowany był przez obiekt klasy `MovieRating` zawierający identyfikator filmu, tytuł oraz ocenę (pole `rating` typu całkowitego).

Procedura pomiarowa

Dla każdego algorytmu i każdego rozmiaru danych wykonano 10 powtórzeń pomiaru. Przed każdym pomiarem dane były kopiowane z oryginalnej tablicy do tablicy pomocniczej, co gwarantowało że każdy algorytm operował na identycznych, nieposortowanych danych wejściowych. Czas sortowania mierzony był za pomocą `std::chrono::steady_clock` z rozdzielczością nanosekund — wyłącznie dla operacji sortowania, bez uwzględnienia czasu kopiowania danych ani obliczania statystyk.

Mierzone wielkości

Dla każdego pomiaru rejestrowano:

- Czas sortowania w nanosekundach
- Średnią arytmetyczną ocen (`avg`)
- Dominantę (modę) ocen (`mean`)

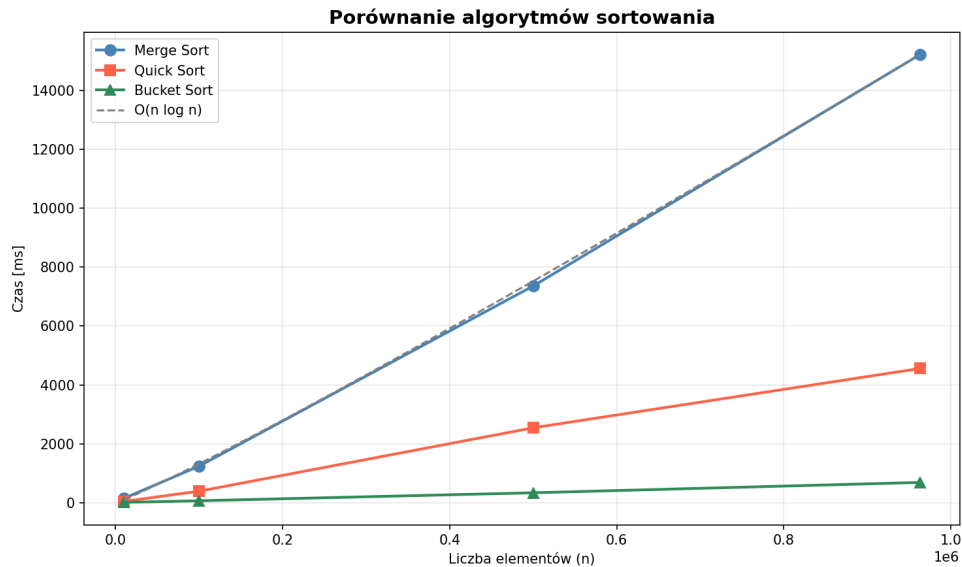
Weryfikacja poprawności zaimplementowanych algorytmów przeprowadzana była automatycznie po każdym wykonaniu sortowania. Wykorzystano w tym celu funkcję standardowej biblioteki C++ `std::is_sorted`, która sprawdza czy elementy tablicy są ułożone w porządku niemalejącym:

Stanowisko badawcze

Eksperymenty przeprowadzono na komputerze wyposażonym w procesor Intel Core i5-620 (architektura 64-bitowa) oraz 8 GB pamięci RAM DDR3 taktowanej z częstotliwością 1600 MT/s. System operacyjny to Kali GNU/Linux 2026.1. Kod źródłowy skompilowano kompilatorem `g++` w wersji 15.2.0-15.

3 Wyniki

Poniższa tabela przedstawia uśrednione czasy sortowania z 10 powtórzeń dla każdego algorytmu i rozmiaru danych.



Rysunek 1: Porównanie czasu sortowania algorytmów

n	Bucket sort [ms]	Quick sort [ms]	Merge sort [ms]
10 000	11,06	44,90	148,30
100 000	65,95	390,65	1 241,02
500 000	336,76	2 543,44	7 358,58
962 903	687,84	4 556,43	15 209,67

Tabela 1: Uśrednione czasy sortowania algorytmów

Statystyki zbioru danych były identyczne niezależnie od zastosowanego algorytmu, co potwierdza poprawność wszystkich implementacji.

n	Średnia ocen	Dominanta
10 000	5,46	1
100 000	6,09	10
500 000	6,67	10
962 903	6,64	10

Tabela 2: Statystyki zbioru danych IMDb

4 Wnioski

Przeprowadzone eksperymenty ujawniły wyraźne różnice w efektywności badanych algorytmów na analizowanym zbiorze danych.

Najszybszym algorytmem okazał się **bucket sort**, który dla 962 903 elementów osią-

gnął czas 687,84 ms. Wynik ten potwierdza teoretyczną złożoność liniową $O(n+k)$, gdzie $k = 11$ odpowiada liczbie możliwych wartości ocen (0–10). Dzięki temu bucket sort wykonuje jedynie trzy liniowe przejścia przez dane: zliczanie wystąpień, wypełnianie kubełków i składanie wyniku — bez żadnych porównań między elementami. Jest to algorytm szczególnie efektywny gdy zakres wartości k jest mały względem liczby elementów n , co w tym przypadku jest spełnione.

Quick sort zajął drugie miejsce z czasem 4 556,43 ms dla pełnego zbioru danych, czyli około 6,6-krotność czasu bucket sortu. Pomimo teoretycznej złożoności $O(n \log n)$, na danych z zaledwie 11 unikalnymi wartościami quicksort wykonuje wiele zbędnych porównań i przestawień elementów o identycznych wartościach. Warto jednak zauważyć że krzywa czasowa quicksorta jest zbliżona do $O(n \log n)$, co potwierdza wykres.

Merge sort okazał się najwolniejszym algorytmem — dla 962 903 elementów osiągnął czas 15 209,67 ms, czyli ponad 22-krotność czasu bucket sortu i ponad 3-krotność czasu quicksorta. Mimo gwarantowanej złożoności $O(n \log n)$, merge sort ponosi znaczny narzut wynikający z konieczności alokacji pomocniczej tablicy przy każdym scalaniu oraz dużej liczby operacji kopiowania danych w pamięci.